Spec2Doc2RTL: RTL Generation from Specification with Natural Language Representation

Zihao Chen[1,†], Zihan Lin[1,†], Xinhua Chen[1,†], Zhiyi Liu[1], Changxu Liu[1], Yuxuan Qiao[1], Yifei Feng[2], Junjie Zuo[1], Yifan Song[1], Fan Yang[1,*]
[1]State Key Lab of Integrated Chips and Systems, School of Microelectronics, Fudan University, Shanghai, China
[2]China and Shanghai Key Laboratory of Data Science, School of Computer Science, Fudan University, Shanghai, China

***Abstract*—This paper presents Spec2Doc2RTL, a hierarchical LLM-based RTL code generation framework leveraging natural language as the intermediate design representation. To address the complexity of design hierarchies and circuit diversity, we propose a universal recursive decomposition method that transforms the circuit design process into the nested module implementations. We introduce a register-transfer-level (RTL) code generation pipeline from design specifications based on natural language representation, which includes two LLM-friendly stages: design document generation (Spec2Doc) and document-to-RTL translation (Doc2RTL). Moreover, a fully automated iterative design workflow is implemented, with script generation, testing, and debugging integrated. Experimental results demonstrate that Spec2Doc2RTL can generate a wide spectrum of circuits, from complex systems like CPUs and NTTs to foundational bottom-level modules with a competitive 78.8% accuracy on the Revisiting VerilogEval benchmark.**

***Index Terms*—RTL generation, large language model, natural language design representation**

I. Introduction

The front-end design of digital circuits, including architecture design and Register Transfer Level (RTL) code generation, heavily relies on manual implementation. These tasks require extensive theoretical knowledge and engineering experience, making standardizing them with fixed rules in traditional design automation tools challenging.

Recent advancements in large language models (LLMs) have opened new possibilities for knowledge-intensive digital circuit design. Powered by abundant data, large-scale trainable parameters, and the multi-head self-attention mechanism [1], LLMs have shown a solid ability to capture long-range dependencies and generate coherent, contextually relevant content [2]–[5].

While general-purpose LLMs like GPT-4 [3] show promise, their limited hardware knowledge constrains their effectiveness in specification-to-RTL (Spec2RTL) tasks. Inspired by LLM applications in software development [6]–[8], researchers have turned to specialized LLMs, training them on Spec-RTL data pairs to generate small RTL modules. DAVE [9] pioneers LLM fine-tuning for RTL generation, with later studies [10]–[17] enhancing training frameworks and datasets. CodeV [18] further advances the field by creating code-based summaries to expand training data, achieving a state-of-the-art 78.1% pass rate on VerilogEval benchmark [16]. ChatCPU [19], meanwhile, tailors the LLM for a six-stage pipeline CPU, guiding it to fill in module code within a predefined architecture.

However, these methods primarily adapt software-oriented LLM approaches to hardware design yet struggle with the inherent data scarcity of RTL corpora, in stark contrast to the vast open-source repositories available in the software domain. Consequently, traditional Spec2RTL fine-tuning approaches are constrained to small-scale circuits [16] or specific large-scale designs (e.g., a particular six-stage pipeline CPU [19]), without adaptability to handle the diverse complexity and functional demands of digital circuits. Specifically, these challenges of hardware design remain unsolved:

Abstraction of RTL representation. Unlike the sequential nature of software code, RTL code relies on concurrency and timing, enabling parallel operations within a single clock cycle and requiring precise synchronization to prevent data conflicts. Such complexity makes RTL fundamentally different and highly abstract for LLMs, which are primarily trained on software data. Consequently, a design representation that is both LLM-compatible and directly mappable to RTL is essential to bridge the gap between specs and RTL code.

Complexity of design levels. Digital circuit systems span multiple design levels, each requiring unique expertise for module structure, interconnections, and timing. Therefore, it is hard to conclude a universal digital circuit flow for LLMs. Current LLM approaches rely on workflows customized for each circuit, reducing adaptability and reusability. A flexible design method applying across various circuits and levels is urgently needed.

Unscalability of manual testbenches. Generating hardware testbenches is inherently challenging for LLMs because they struggle to identify circuit functionality, specify typical test stimuli, and calculate the corresponding output signal values. Current approaches often rely on predefined, static testbenches, limiting their applicability to diverse circuit designs and configurations. Efforts to automate testbench generation have been minimal. AutoBench [20] is the first LLM-based testbench generator for digital circuit design, requiring only the description of the design under test to automatically generate comprehensive testbenches. In AutoBench, a hybrid testbench structure and a self-checking system are implemented using LLMs. However, there remains a lack of a function-level design-verification framework that fully automates both RTL code and testbench generation, freeing manual labor.

To address the challenges described above, this paper proposes Spec2Doc2RTL, an end-to-end, hierarchical LLM-based RTL generation framework that uses natural language as the intermediate design representation. Detailed contributions are as follows:

- A natural language design representation is proposed to avoid the abstract RTL representation for LLMs. The difficult Spec2RTL task is thus decomposed into two LLM-friendly stages: design documentation completion (Spec2Doc) and design representation translation (Doc2RTL). With all essential design details embedded in the documentation, general-purpose LLMs can then efficiently translate it into RTL code.
- A recursive design decomposition method is proposed to address the complexity of multiple levels for LLMs in digital circuit design. This approach transforms hierarchical, varied design processes into recursive implementations of a single design task, enabling a single LLM to handle multi-level designs across various circuits.
- An automated function-level design-verification framework is proposed to enable LLM to not only generate RTL code given comprehensive document, but also generate testbench automatically and perform function-level debugging. The testbench generation is based on AutoBench [20].

†These authors contributed equally to this paper.
*Corresponding authors: yangfan@fudan.edu.cn.

Experimental results demonstrate that Spec2Doc2RTL significantly surpasses existing Spec2RTL methods on VerilogEval. Moreover, Spec2Doc2RTL has shown capabilities that current methods lack, including designing various complex, large-scale digital circuits (e.g., NTT, CPU) and automatically generating and debugging testbenches.

The remainder of this paper is organized as follows. Section II shows prior knowledge. Section III-D proposes our methods. Section IV presents the experimental results. Section V concludes this paper.

II. Preliminaries

A. LLM Agent

Multiple LLMs, usually state-of-the-art general-purpose LLMs like GPT-4 [3], are frequently organized as autonomous *agents* to solve complex problems collaboratively. Such multi-agent systems enhance the efficiency and scalability of LLMs by enabling interaction, task decomposition, and specialization.

B. LLM Reasoning

Recent advancements in LLM reasoning have significantly enhanced the ability of LLMs to tackle complex tasks by structuring their problem-solving processes. Methods such as chain of thought (CoT) [21] and graph of thought (GoT) [22] enable models to reason step-by-step, explore multiple decision paths, and evaluate interconnected possibilities, respectively. These techniques guide LLMs to solve problems systematically, improving logical consistency and adaptability in scenarios requiring intricate analysis.

C. Retrieval-augmented Generation

Retrieval-augmented generation (RAG) [23] enhances LLMs' performance by combining their generative capabilities with retrieval from external knowledge bases. By dynamically querying domain-specific knowledge bases using vector-based retrieval techniques, RAG integrates precise, context-relevant information into the model's reasoning process.

III. Proposed Approaches

A. Overall Workflow

The workflow of Spec2Doc2RTL is summarized in Fig. 1. It is a multi-agent recursive framework that realizes hierarchical decomposition for complex digital circuit design (also compatible with the design of small circuits), and uses natural language as the intermediate representation for RTL generation tasks from specs. In the Spec2Doc stage, given user-defined specs, with the extracted knowledge through RAG, agent_A follows a scalable CoT to generate detailed function implementation steps, timing, connections, and specs of sub-modules, recording them in a design document of the current module. Then the submodules will be recursively processed. Finally, a design document tree will grow up and then be passed to agent_B. In the Doc2RTL stage, with the help of knowledge base and enhanced AutoBench [20], RTL code for each module will be generated, organized, and verified.

B. Natural Language Design Representation

We transform the process of digital circuit design into a workflow that begins with the generation of *design documents* and culminates in the creation of *RTL code*. With *natural language* intermediate representation, the framework bridges the gap between intuitive natural language descriptions and abstract RTL code, avoiding the bottlenecks commonly associated with direct translation from specs to RTL code. As depicted in Fig. 1, two key stages are organized:

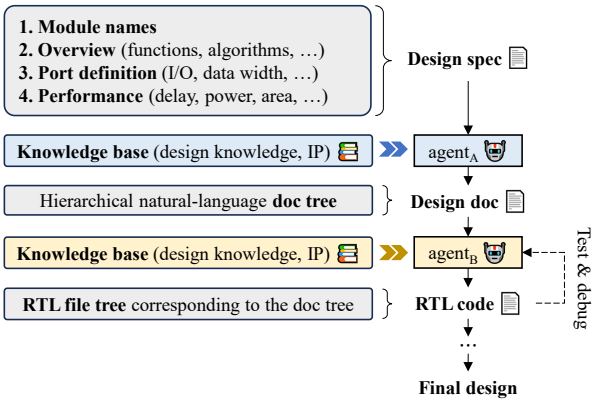


Fig. 1. **The Overall Workflow of the Spec2Doc2RTL Framework**

1) Spec2Doc: translating design specs into documents: This stage utilizes natural language inputs to describe each hierarchical design node based on the recursive decomposition method, which will be presented in detail in Section III-D. Specifically, agent_A is tasked with processing the specs of each design node and generating a comprehensive design document. To ensure completeness and consistency, a template in Fig. 2 is constructed, which standardizes the representation of design documents.

Design Document Template for Each Module

1. Module overview: the name, function, and performance requirements of the module.
2. Functional details: basic description including I/O, primary logic, and customizations.
3. Port definition: the I/O signals of the module, detailing the signal names, directions, bit widths, and their corresponding meanings.
4. Parameter definition: parameters such as data width and others.
5. Specs for submodules: If the current module requires further decomposition, list all submodules with their specs for items **1-4**.
6. Submodule interconnections: connections among the interfaces of the submodules.
7. Submodule timings: the timing requirements within the module and between submodules, including details on clock domains, synchronous/asynchronous considerations.
8. Referenced knowledge entries: the design knowledge retrieved for the current design node.

Fig. 2. **Design Document Template for Each Module (Design Node)**

Fig. 3 shows the CoT of agent_A for each design node. Agent_A is guided to generate documents step by step, according to the CoT and knowledge base, which is prepared by experts using open-source code, textbooks, and papers, and is equipped for retrieval in a knowledge base or prompt.

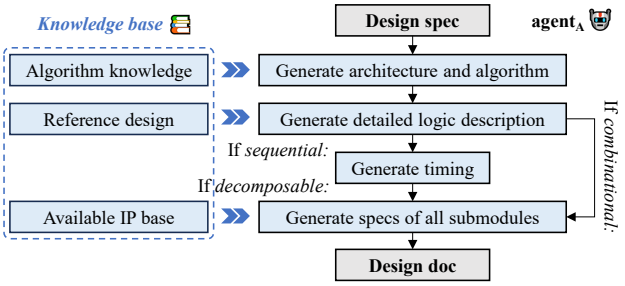


Fig. 3. **The CoT of agent_A for Spec2Doc in Processing Each Module**

Based on the provided specs, agent_A evaluates the design requirements and performance criteria, querying the knowledge base for relevant matches. This step involves generating appropriate algorithms and architectures, with the extracted knowledge subsequently passed to agent_A. Next, the knowledge base of design experiences is consulted

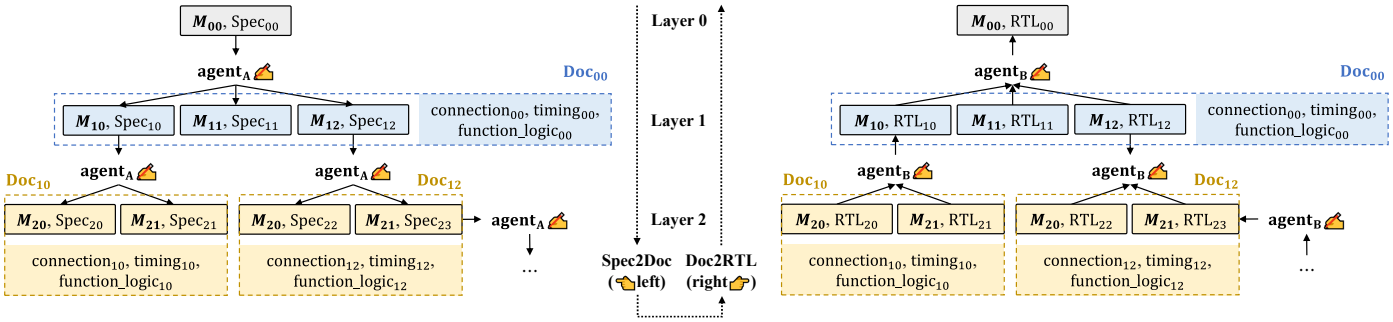


Fig. 4. **The Recursive Design Decomposition Method in Spec2Doc2RTL**

to identify historical implementations with similar structures or functionalities, guiding agent$_A$ to generate detailed steps for functional implementation. Following this, for the sequential logic circuit the temporal relationships are derived based on the interdependencies of the functional implementation steps. Afterwards, agent$_A$ assesses, based on its internal knowledge and considerations of complexity and reusability, whether the module requires further decomposition into smaller submodules. If decomposition is necessary, the module is divided into multiple smaller, functionally independent submodules. For each submodule, the IP library is searched to determine if parameterized IP components exist that can fulfill the required functionality. If no such IP exists, the framework generates the specifications for the submodules, including their names, interfaces, and functional summaries. The output of each step is formatted according to the template above and exported as a *.json* file. When the recursion finishes, a design document tree will be generated.

2) Doc2RTL: translating document into RTL code: In this stage, the generated design document is mapped into a tree-structured RTL code. To ensure the correctness of this mapping, the Doc2RTL process is mainly driven by agent$_B$ (specifically agent$_{B1}$ and agent$_{B2}$). Design documents are systematically translated into verified RTL code by iteratively checking each design level from the top module to leaf nodes (see Section IV). The knowledge base includes specific hardware implementation methods for each sub-module, ensuring consistency between the documentation and the resulting RTL code.

Fig. 5 presents the CoT of agent$_B$ for each design node. Agent$_{B1}$ first searches the RAG to gather necessary knowledge, including algorithm details and example code. It then begins coding following its detailed CoT and debugging for syntax errors. To automate function-level debugging, we implement agent$_{B2}$ to automatically generate a testbench for function verification.

C. Function-level Design-verification

As is shown in Fig. 5, the workflow of Doc2RTL mimics human behavior in RTL coding and testing. When assigned a task, agent$_{B1}$ first evaluates whether the task description is detailed enough for coding by summarizing all the gathered information according to a template, including algorithm details and example code. If not, it calls the RAG to gather the missing knowledge. It then begins coding and debugging for syntax errors with the help of Icarus Verilog [24], a popular open-source Verilog simulation and synthesis tool. Additionally, to automate function-level debugging, agent$_{B2}$ automatically generate a testbench for function verification based on Autobench [20].

D. Recursive Design Decomposition

We propose a recursive design decomposition method for hierarchical digital circuits leveraging LLMs. Digital circuits are inherently

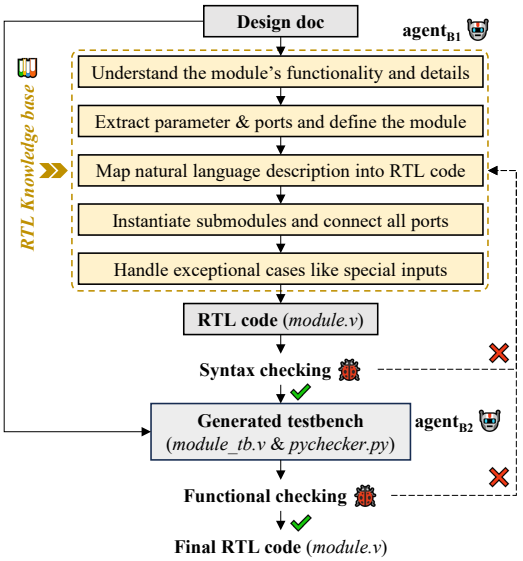


Fig. 5. **The CoT of agent$_B$ for Doc2RTL in Processing Each Module**

hierarchical, posing challenges in both structural complexity and functional clarity. By employing a multi-agent framework, the design task is broken down into smaller sub-tasks, avoiding the inefficiencies of traditional modular design and adhering to a scalable tree structure.

As shown in the left of Fig. 4, each circuit module is treated as a node within a hierarchical tree. For each node in the tree, the LLM-based agent processes design specs (e.g., name, functional overview, and interface constraints) and leverages the capabilities of LLM itself and RAG strategy to generate a detailed document, which includes the specifications of sub-modules, connection, timing and functional logic. Then agent$_A$ recursively continues to process the specs of sub-modules. Agent$_A$ systematically proceeds from top-level modules to lower levels, iteratively refining the design specification until no further decomposition is possible. By assigning agent$_A$ to each design node, the system optimally manages the design process for the current node M$_{nk}$ using its specifications Spec$_{nk}$. Leveraging RAG technique, the agent fetches and integrates relevant knowledge K$_{nk}$ for each sub-task, ensuring alignment with design goals and consistency in modular connectivity and timing relationships. The process is formalized as follows:

$$\{\text{subSpecs}, \text{conn}, \text{time}, \text{func}\}_{nk} = \text{agent}_A(\text{K}_{nk}, \text{Spec}_{nk}), \quad (1)$$

where subSpec, conn, time and func represent the sub-specifications,

interconnections, timing relationships and functional logic of current node generated for the given design task, respectively. The recursive algorithm, outlined in Algorithm 1, begins with the top-level module specifications.

Algorithm 1 Recursive design document generation algorithm

Input: Top-level design requirements topSpec, knowledge base K
Output: Specs, interconnections, timing, and function logic
define design_tree_growth ($\text{Spec}_{\text{node}}$):
  fetch: $\text{K}_{\text{node}} = \text{RAG_fetch}\ (\text{Spec}_{\text{node}}, \text{K})$
  grow: $\{\text{subSpecs}, \text{conn}, \text{time}, \text{func}\} = \text{agent}_{\text{A}}(\text{K}_{\text{node}}, \text{Spec}_{\text{node}})$
  for subSpec in subSpecs:
    if subSpec is **not** a basic module:
      design_tree_growth (subSpec)
    else:
      output_final_design_description (subSpec)
  return {subSpecs, conn, time, func}
design_tree_growth (topSpec)

As shown in the right of Fig. 4, after the recursion of Spec2Doc, the design tree of documents will be passed to the agent for Doc2RTL. Unlike the decomposition of modules, the generation of RTL code follows a bottom-up backtracking process. Agent_{B} starts from the lowest-level sub-modules. Once all sub-modules at a given hierarchical level have been successfully generated, the process progresses upward to the next higher-level design node. This backtracking continues until the RTL code for the top-level module is generated and verified for correctness. The hierarchy methodology enhances the traceability of the design process and reduces the complexity of utilizing large language models for digital circuit design.

IV. Experiments

A. Experimental Settings

Our pipeline for each design node consists of 2 stages, Spec2Doc and Doc2RTL, with independent LLM-based agents handling tasks respectively. Due to the extraordinary capabilities of comprehension, information processing, and reasoning, we use GPT-4 Turbo [3] to drive the agents. To evaluate the performance of our Spec2Doc2RTL framework on Verilog code generation from small modules to large-scale system circuits, we use a representative benchmark, VerilogEval [17], to evaluate its competence in RTL generation for general small-scale circuits and two relatively large-scale circuits to demonstrate its capability to handle complex circuits.

We follow the original paper [16] and use the metric $pass@k$, which is defined as the likelihood of the top-k LLM-generated modules will pass the testbench:

$$\text{pass@k} = E_i \left(1 - \frac{\binom{n-c_i}{k}}{\binom{n}{k}} \right), \tag{2}$$

where n is the number of trials for each task, c_i is the number of correct attempts that pass testing, and k is the success threshold.

B. Small Circuit Design Results

We compare the performance of our framework on the small circuit design using Revisiting VerilogEval Benchmark [17] with other models with different sizes and different types, measured by the benchmark pass rate pass@1. For each model, we set *temperature* = 0 and *top_p* = 0.01 to ensure stable output. Table I shows the comparison with relevant RTL generation solutions. These models include general-purpose LLMs, coding LLMs and LLMs tailored for Verilog generation tasks. Spec2Doc2RTL outperforms all these baselines on this benchmark. Spec2Doc2RTL achieves the highest pass rate at approximately 78.8% ($n = 1$) for zero-shot, surpassing the previously established state-of-the-art frontier of 60.3% ($n = 1$) by GPT-4 Turbo [3]. The pass rates of our framework for one-shot are 76.3% ($n = 1$), surpassing the previously highest pass rates of 58.3% ($n = 1$) by GPT-4 Turbo [3]. The hierarchical design framework with natural language representation shows strong competence in small-scale circuit design. Specifically, we found that regardless of the base model, the highest pass rate of CodeV [18] is only 47.4%. The results in Table I differ from those in [18] because the benchmark version used here is VerilogEval [17]rather than the original version [16].

C. Complicated Circuit Design Results

Besides the brilliant performance with simple circuits, our framework is also able to design more complex circuits. We evaluate this capability with an NTT circuit and a RISC-V CPU.

NTT The Number Theoretic Transform (NTT) is an efficient optimization algorithm for polynomial multiplication, a variant of the Fast Fourier Transform (FFT). To implement a multi-points NTT, it involves in modular operation, butterfly units, and complex connections of modules, which is difficult to handle for other works focusing on RTL generation. Benefiting from our recursive decomposition framework and the intermediate representation of natural language, supported by knowledge bases and chain-of-thought reasoning, our work effectively addresses the complex design requirements for NTT circuits. It is a 16-point NTT module and specific requirements are proposed for efficient hardware resource utilization. Fig. 8 shows the procedure of NTT design flow. Our framework can easily handle the case without mistakes, while GPT-4 Turbo and other works can hardly give a complete and right Verilog code.

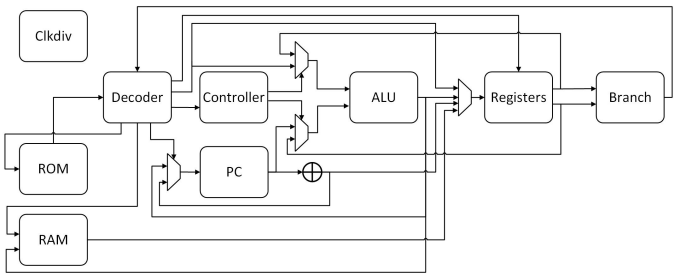


Fig. 6. **Interconnections of submodules in the CPU**

RISC-V CPU A simple RISC-V CPU comprises a decoder, controller, program counter, ALU, branch decider, registers, and several multiplexers, all connected to peripherals like RAM and ROM. The most complex module is the ALU, including an adder, multiplier, divider, and other submodules. The decoder involves intricate connections directly proportional to the instruction set, posing a significant challenge for RTL-coding LLMs. However, with our private RAG, our Spec2Doc and Doc2RTL frameworks successfully manage these challenging tasks through effective decomposing and coding. Fig. 7 shows the procedure for ALU design. For the ALU, complex multipliers and dividers can be designed using IP cores from our private RAG. The decoder also performs well with the example code for part of the instruction set. We emphasize that these approaches—designing with IP cores and example codes—are exactly how humans design complex digital circuits. Fig. 6 shows the interconnections of modules of CPU.

The NTT case shows our framework's ability to extract effective knowledge from the knowledge base to enhance its reasoning and coding capabilities and deal with hierarchical design requirements. In the CPU case, which involves more complicated connections and hierarchy between modules, our framework still manages to handle the design, showcasing its great potential for automated circuit design.

TABLE I
PERFORMANCE COMPARISON OF RTL CODE GENERATORS ON VERILOGEVAL [17] BENCHMARK

Model Name	Model Size	Type	Task: Specification-to-RTL	
			0-Shot (n=1)	1-Shot (n=1)
GPT-4 Turbo [3]	N/A	General	60.3%	58.3%
GPT-4 [3]	N/A	General	44.2%	50.6%
Mistral Large [26]	N/A	General	35.3%	48.7%
Llama 3.1 [5]	405B	General	58.8%	57.7%
Llama 3.1 [5]	70B	General	37.8%	48.7%
Llama 3 [5]	70B	General	41.7%	37.2%
CodeLlama [27]	70B	Coding	34.0%	41.0%
DeepSeek Coder [28]	33B	Coding	17.3%	37.2%
Llama 3.1 [5]	8B	General	20.5%	26.9%
CodeGemma [29]	7B	Coding	21.0%	23.1%
DeepSeek Coder [28]	6.7B	Coding	28.2%	26.9%
RTL-Coder (corr.) [13]	6.7B	Verilog RTL	35.3%	35.3%
RTL-Coder (orig.) [13]	6.7B	Verilog RTL	36.5%	35.3%
CodeV (CodeQwen-1.5) [13]	7B	Verilog RTL	37.2%	47.4%
CodeV (CodeLlama) [13]	7B	Verilog RTL	28.2%	28.2%
CodeV (DeepSeek Coder) [13]	6.7B	Verilog RTL	8.3%	45.5%
Spec2Doc2RTL (GPT-4 Turbo)	**N/A**	**Verilog RTL**	**78.8%**	**76.3%**

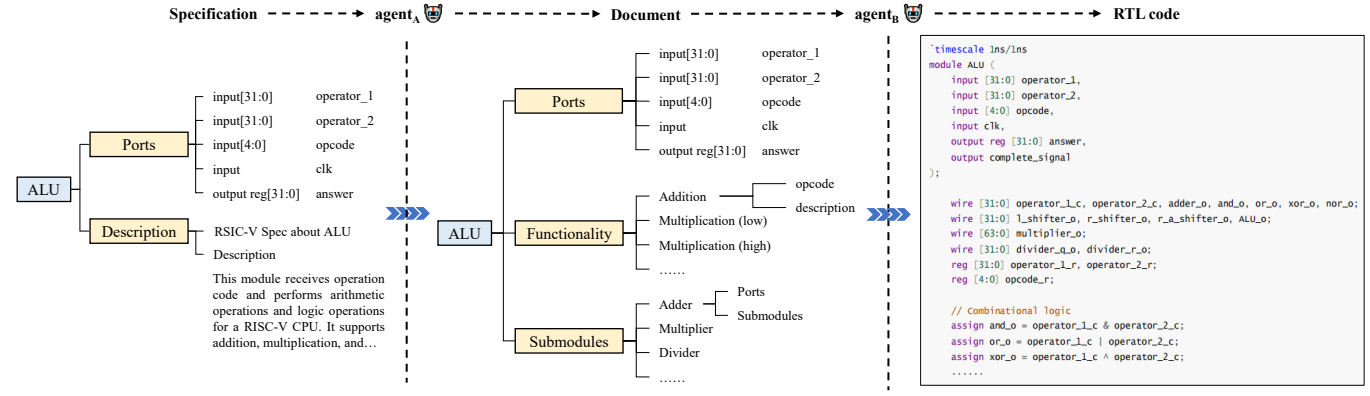


Fig. 7. **ALU design flow in the Spec2Doc2RTL framework**

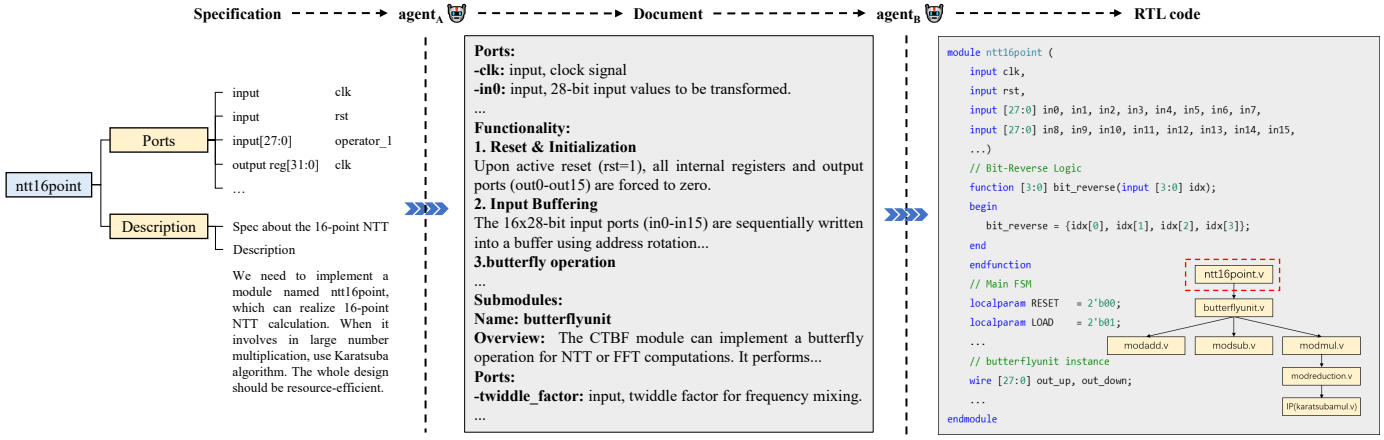


Fig. 8. **NTT design flow in the Spec2Doc2RTL framework**

V. CONCLUSIONS

This paper introduces Spec2Doc2RTL, a novel framework that bridges the gap from specs to RTL code using LLMs based on the natural language representation. By employing a recursive decomposition methodology, the framework is able to decompose complex circuits into a design node tree. For each node, our pipeline has two stages: Spec2Doc, which translates high-level specs into detailed design documents, and Doc2RTL, which converts documents into verified codes. Experimental results shows its better performance on the Revisiting VerilogEval benchmark and its potential in designing a wide range of circuits, from small modules to complex systems. Spec2Doc2RTL not only surpasses traditional approaches but also shows a new method for automating and scaling RTL generation tasks.

ACKNOWLEDGMENT

This research is supported partly by National Natural Science Foundation of China (NSFC) research projects 61929102, 62141407 and 62090025, partly by Science and Technology Commission of Shanghai Municipality Project (22DZ1101100).

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[2] L. Floridi and M. Chiriatti, "Gpt-3: Its nature, scope, limits, and consequences," *Minds and Machines*, vol. 30, pp. 681–694, 2020.

[3] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[4] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[5] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[6] A. M. Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, M. C. Desmarais, and Z. M. J. Jiang, "Github copilot ai pair programmer: Asset or liability?" *Journal of Systems and Software*, vol. 203, p. 111734, 2023.

[7] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim *et al.*, "Starcoder: may the source be with you!" *arXiv preprint arXiv:2305.06161*, 2023.

[8] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago *et al.*, "Competition-level code generation with alphacode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.

[9] H. Pearce, B. Tan, and R. Karri, "Dave: Deriving automatically verilog from english," in *Proceedings of the 2020 ACM/IEEE Workshop on Machine Learning for CAD*, 2020, pp. 27–32.

[10] S. Thakur, B. Ahmad, Z. Fan, H. Pearce, B. Tan, R. Karri, B. Dolan-Gavitt, and S. Garg, "Benchmarking large language models for automated verilog rtl code generation," in *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2023, pp. 1–6.

[11] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, "Verigen: A large language model for verilog code generation," *ACM Transactions on Design Automation of Electronic Systems*, vol. 29, no. 3, pp. 1–31, 2024.

[12] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "Rtllm: An open-source benchmark for design rtl generation with large language model," in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2024, pp. 722–727.

[13] S. Liu, W. Fang, Y. Lu, Q. Zhang, H. Zhang, and Z. Xie, "Rtlcoder: Outperforming gpt-3.5 in design rtl generation with our open-source dataset and lightweight solution," in *2024 IEEE LLM Aided Design Workshop (LAD)*. IEEE, 2024, pp. 1–5.

[14] K. Chang, K. Wang, N. Yang, Y. Wang, D. Jin, W. Zhu, Z. Chen, C. Li, H. Yan, Y. Zhou *et al.*, "Data is all you need: Finetuning llms for chip design via an automated design-data augmentation framework," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.

[15] Z. Pei, H.-L. Zhen, M. Yuan, Y. Huang, and B. Yu, "Betterv: Controlled verilog generation with discriminative guidance," *arXiv preprint arXiv:2402.03375*, 2024.

[16] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "Verilogeval: Evaluating large language models for verilog code generation," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–8.

[17] N. Pinckney, C. Batten, M. Liu, H. Ren, and B. Khailany, "Revisiting verilogeval: Newer llms, in-context learning, and specification-to-rtl tasks," *arXiv preprint arXiv:2408.11053*, 2024.

[18] Y. Zhao, D. Huang, C. Li, P. Jin, Z. Nan, T. Ma, L. Qi, Y. Pan, Z. Zhang, R. Zhang *et al.*, "Codev: Empowering llms for verilog generation through multi-level summarization," *arXiv preprint arXiv:2407.10424*, 2024.

[19] W. Xi, W. Gwok-Waa, W. Sam-Zaak, Z. Layton, L. Tianyang, T. Qi, and Y. Jianmin, "Chatcpu: An agile cpu design & verification platform with llm," in *Proceedings of the 61th DAC*, 2024.

[20] R. Qiu, G. L. Zhang, R. Drechsler, U. Schlichtmann, and B. Li, "Autobench: Automatic testbench generation and evaluation using llms for hdl design," in *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD*, 2024, pp. 1–10.

[21] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.

[22] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi, J. Gajda, T. Lehmann, H. Niewiadomski, P. Nyczyk *et al.*, "Graph of thoughts: Solving elaborate problems with large language models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, 2024, pp. 17 682–17 690.

[23] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[24] S. Williams and M. Baxter, "Icarus verilog: open-source verilog more than a year later," *Linux Journal*, vol. 2002, no. 99, p. 3, 2002.

[25] N. Pinckney, C. Batten, M. Liu, H. Ren, and B. Khailany, "Revisiting verilogeval: Newer llms, in-context learning, and specification-to-rtl tasks," *arXiv preprint arXiv:2408.11053*, 2024.

[26] D. McDonald, R. Papadopoulos, and L. Benningfield, "Reducing llm hallucination using knowledge distillation: A case study with mistral large and mmlu benchmark," *Authorea Preprints*, 2024.

[27] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez *et al.*, "Code llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.

[28] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. Li *et al.*, "Deepseek-coder: When the large language model meets programming–the rise of code intelligence," *arXiv preprint arXiv:2401.14196*, 2024.

[29] C. Team, H. Zhao, J. Hui, J. Howland, N. Nguyen, S. Zuo, A. Hu, C. A. Choquette-Choo, J. Shen, J. Kelley *et al.*, "Codegemma: Open code models based on gemma," *arXiv preprint arXiv:2406.11409*, 2024.